



01 — How Websites Work



Introduction

Before learning how to build websites, I wanted to understand what actually happens when someone visits a website.

When I type a website address such as:

```
https://www.example.com
```

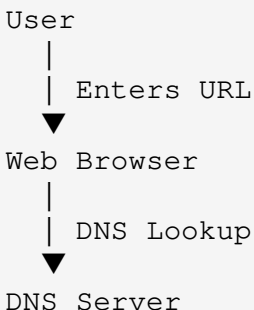
into a web browser and press **Enter**, a series of processes happen in the background before the webpage appears on my screen.

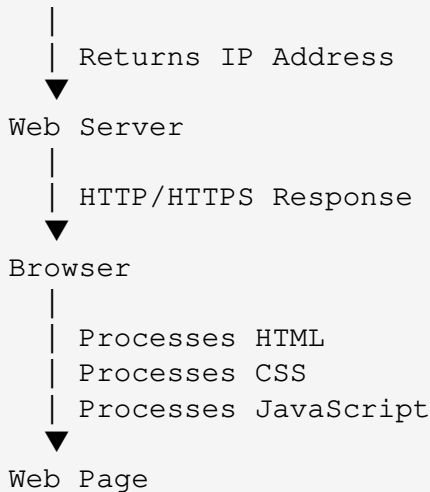
This is my documentation of what I have learned.



The Basic Process

At a simplified level:





1. The User Enters a URL

Everything starts when a user enters a website address into a browser.

For example:

```
https://www.example.com
```

A URL is a **Uniform Resource Locator**.

It tells the browser where a resource is located on the internet.

A URL can contain several parts:

```
https://www.example.com/about
|           |           |
|           |           | Path
|           |           | Domain
|           |           | Protocol
```

2 The Browser Interprets the URL

The browser needs to determine where the requested website is located.

For a domain such as:

```
www.example.com
```

the browser needs an IP address to communicate with the destination server.

This is where **DNS** becomes important.

3 DNS Finds the IP Address

DNS stands for:

Domain Name System

Humans prefer names such as:

```
example.com
```

Computers communicate using IP addresses such as:

```
93.184.216.34
```

DNS acts like a directory that helps translate a domain name into an IP address.

Simplified:

```
example.com
  ↓
DNS
  ↓
93.184.216.34
```

The browser can then use the IP address to communicate with the appropriate server.

4📖 The Browser Connects to the Server

After finding the destination IP address, the browser establishes communication with the server.

Depending on the website, this communication can involve:

- TCP
- TLS
- HTTPS

For a secure HTTPS connection, TLS helps encrypt communication between the browser and the server.

5📖 The Browser Sends a Request

The browser sends an HTTP request to the server.

A simplified request might look like:

```
GET / HTTP/1.1  
Host: example.com
```

The browser is essentially saying:

"I would like to retrieve this resource."

6 The Web Server Processes the Request

The request reaches a web server.

The server may:

- Find a requested file
- Run application code
- Query a database
- Authenticate a user
- Process information
- Generate a response

For a simple website, the server may return an HTML document.

For a more complex application, the server might perform many operations before responding.

7 The Server Sends a Response

The server sends an HTTP response back to the browser.

A response can contain:

- HTML
- CSS
- JavaScript
- Images
- JSON
- Other resources

A simplified response might look like:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

The `200 OK` status indicates that the request was successfully processed.

The Browser Receives the HTML

The browser receives the HTML document.

For example:

```
<!DOCTYPE html>

<html>
<head>
  <title>My Website</title>
</head>

<body>
```

```
<h1>Hello World!</h1>

</body>
</html>
```

The browser reads the HTML and begins constructing the webpage.

9📄 The Browser Loads Other Resources

The HTML may reference other files.

For example:

```
<link rel="stylesheet" href="style.css">

<script src="script.js"></script>


```

The browser may then request these additional resources from the server.

So loading a webpage can involve many requests.

```
HTML
├──
├── CSS
├── JavaScript
├── Images
├── Fonts
└── Other resources
```

CSS Controls the Appearance

CSS stands for:

Cascading Style Sheets

HTML provides structure.

CSS controls appearance.

For example:

```
<h1>Hello World!</h1>
```

HTML creates the heading.

CSS can control:

- Color
- Size
- Spacing
- Position
- Layout
- Fonts
- Animations

JavaScript Adds Behaviour

JavaScript allows webpages to become interactive and dynamic.

For example:

- Buttons can respond to clicks
- Menus can open and close
- Forms can be validated
- Content can change without refreshing the page
- Websites can communicate with APIs

A simple example:

```
document.querySelector("button").addEventListener("click",  
    alert("Hello!");  
});
```

The Browser Renders the Page

After receiving and processing the required resources, the browser renders the webpage.

The simplified relationship is:

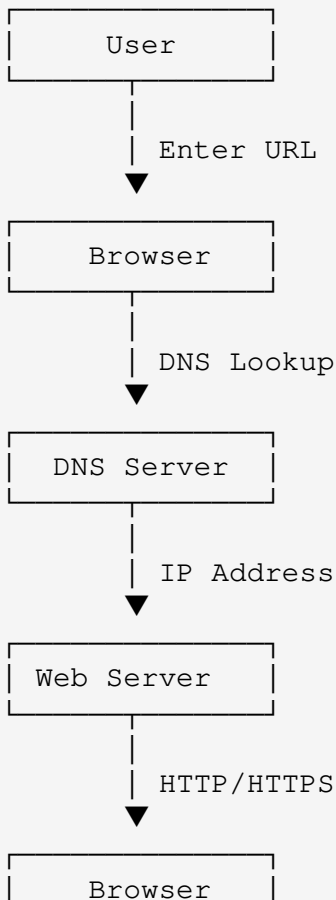
```
HTML  
↓  
Structure  
  
CSS  
↓  
Appearance  
  
JavaScript  
↓  
Behaviour  
  
↓
```

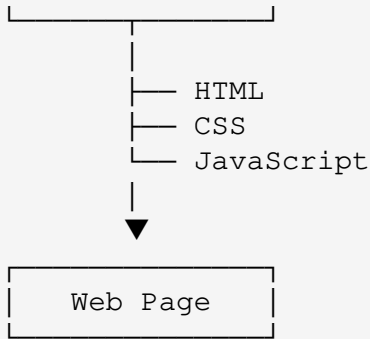
Webpage

The result is what the user sees and interacts with.

The Complete Journey

Putting everything together:





What I Learned

From this topic, I learned that visiting a website is not simply:

Type URL → Website appears

There are several processes happening behind the scenes.

The simplified process is:

```
URL
↓
Browser
↓
DNS
↓
IP Address
↓
Server
↓
HTTP/HTTPS Request
↓
Server Response
```

↓
HTML
↓
CSS + JavaScript + Resources
↓
Browser Rendering
↓
Web Page

Key Terms

Term	Meaning
URL	Address used to locate a resource
Browser	Software used to access and display web content
DNS	System that translates domain names into IP addresses
IP Address	Address used to identify a device/network destination
Server	Computer/system that provides resources or services
HTTP	Protocol used for communication on the web
HTTPS	HTTP secured using TLS
HTML	Provides webpage structure
CSS	Controls webpage presentation
JavaScript	Adds behaviour and interactivity

? Questions I Want to Explore Next

Understanding how websites work has raised some new questions for me:

- How exactly does DNS find an IP address?
- What is a client?
- What is a server?
- How does HTTP actually work?
- What happens inside a browser?
- What is web hosting?
- What is the difference between frontend and backend?
- What happens when a website uses a database?

These will become the next topics in my Web Fundamentals journey.

Progress

Stage: Web Fundamentals

Topic: How Websites Work

Status: ● Completed

Next: Client-Server Model

The more I understand what happens behind the screen, the better I can understand what I'm building.